

AMRITA VISHWA VIDYAPEETHAM

Department of Computer Science and Engineering

Machine Learning Capstone Project

Review 1: Regression and Classification Analysis

Course Code: 23CSE301

A Comprehensive Technical Report on the

*California Housing Regression Track and
Adult / Census Income Classification Track (Part A)*

Repository: Review_1_ASV_reg_cla

Author (GitHub): ashwkumarsrinialg-cpu

Submitted by

Ashwin Kumar Srinivasan CB.SC.U4CSE24607

Salla Sai Sumallik CB.SC.U4CSE24647

Tallada Vamshidhar CB.SC.U4CSE24660

B.Tech. Computer Science and Engineering – III Year

Academic Year 2026–27

September 6, 2026

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Purpose and Scope of this Report	1
1.3	Repository Structure	2
1.3.1	Software Environment and Dependencies	2
1.3.2	Execution Workflow	3
1.4	Course Context: The 23CSE301 Capstone Rubric	3
2	Datasets	4
2.1	California Housing Dataset (Regression Track)	4
2.1.1	Description	4
2.1.2	Data Quality Considerations	5
2.2	Adult / Census Income Dataset (Classification Track)	5
2.2.1	Description	5
2.2.2	Data Quality Considerations	5
2.3	Why These Two Datasets?	6
3	Regression Track: California Housing Price Prediction	7
3.1	Problem Statement	7
3.2	Exploratory Data Analysis	7
3.3	Data Preprocessing	8
3.4	Feature Engineering	9
3.5	Algorithms Implemented	9
3.5.1	Algorithmic Rationale	10
3.6	Evaluation Metrics	10
3.7	Regression Results	11
3.7.1	Interpretation of Baseline Results	11
3.8	Cross-Validation	12
3.9	Hyperparameter Tuning	13
3.9.1	Tuned Random Forest Regressor	13
3.9.2	Tuned Gradient Boosting Regressor	13
3.10	Final Model Selection	14
3.11	Model Diagnostics	14
4	Classification Track – Part A: Income Prediction	16
4.1	Problem Statement	16
4.2	Exploratory Data Analysis	16
4.3	Data Preprocessing	17
4.4	Feature Engineering	18

4.5	Algorithms Implemented	18
4.5.1	Algorithmic Rationale	18
4.6	Evaluation Metrics	19
4.7	Classification Results	19
4.7.1	Interpretation of Results	19
4.8	Best Model Selection	20
4.8.1	Class-wise Performance of the Best Model	21
4.9	Additional Diagnostic Outputs	21
5	Cross-Track Summary and Model Selection	22
5.1	Consolidated Model Selection	22
5.2	Common Methodological Threads	22
5.3	Reproducibility and Data Leakage Prevention	23
5.4	Capstone Requirements Coverage	23
5.5	Review 2: Planned Work	24
5.5.1	Classification Part B	24
5.5.2	Full Clustering Track	25
5.5.3	Final Documentation	25
5.6	Optional Future Extensions	25
6	Theoretical Background of Algorithms Used	27
6.1	Linear-Family Regression Models	27
6.1.1	Ordinary Least Squares (Linear Regression)	27
6.1.2	Ridge, Lasso, and ElasticNet	27
6.1.3	Polynomial Regression	27
6.2	Tree-Based Models	27
6.2.1	Decision Trees	28
6.2.2	Random Forest	28
6.2.3	Gradient Boosting	28
6.3	Instance-Based and Kernel Methods	28
6.3.1	K-Nearest Neighbors	28
6.3.2	Support Vector Machines (SVR and SVC)	28
6.4	Probabilistic Models	29
6.4.1	Gaussian Naive Bayes	29
6.4.2	Logistic Regression	29
6.5	Evaluation Concepts	29
6.5.1	R^2 , RMSE, and MAE	29
6.5.2	Precision, Recall, F1, and ROC-AUC	29
6.5.3	Cross-Validation and Grid Search	29
7	Conclusion	31
7.1	Summary of Findings	31
7.2	Methodological Strengths	31
7.3	Limitations and Areas for Improvement	32
7.4	Outlook Toward Review 2	32
	References	33

List of Tables

1.1	Project dependencies as declared in <code>requirements.txt</code>	2
2.1	California Housing dataset summary	4
2.2	Adult / Census Income dataset summary	5
3.1	Baseline test-set performance of all ten regression algorithms	11
3.2	5-fold cross-validation results for the two strongest ensemble models	12
3.3	Tuned ensemble model performance on the held-out test set	13
4.1	Test-set performance of all five Classification Part A algorithms	20
4.2	Per-class classification report for the selected SVC model	21
5.1	Consolidated model selection summary across both tracks	22
5.2	Review 1 capstone requirements coverage	24

Chapter 1

Introduction

1.1 Project Overview

This report presents a comprehensive technical analysis of the **Review 1** submission for the **23CSE301 Machine Learning Capstone Project** (Academic Year 2026–27), hosted in the GitHub repository `Review_1_ASV_reg_cla`. The capstone requires students to demonstrate mastery of the complete supervised-learning pipeline – from raw data ingestion through exploratory data analysis, cleaning, pre-processing, feature engineering, model training, systematic comparison, hyperparameter optimization, and final model serialization – across three canonical problem types: regression, classification, and clustering.

At the Review 1 checkpoint, the project scope is deliberately narrowed to two of the three tracks:

- (i) A **complete regression track** built on the well-known California Housing dataset, in which all ten regression algorithms mandated by the course rubric are implemented, evaluated, cross-validated, and tuned.
- (ii) The **first half of the classification track** (“Part A”), built on the Adult / Census Income dataset, in which five classification algorithms are implemented and compared.

The remaining classification algorithms (Part B) and the entire clustering track are explicitly deferred to Review 2, as documented in the repository’s roadmap. This staged delivery mirrors standard industry and academic practice, where a capstone project is decomposed into incremental milestones so that a working, evaluable subset of functionality is always available for review.

1.2 Purpose and Scope of this Report

The purpose of this document is to provide an independent, structured, and pedagogically thorough account of everything contained in the repository: its structure, its datasets, the theoretical basis of every algorithm employed, the preprocessing and feature-engineering decisions made, the experimental results obtained, and the reasoning behind final model selection. Where the repository’s own documentation (the `README.md` file) reports a result or a design decision, this report reproduces and explains that result, situates it in the broader context of machine-learning methodology, and, where useful, adds conceptual background so that a reader unfamiliar with a particular algorithm or metric can still follow the analysis.

This report is organized so that it can serve three simultaneous purposes:

- As a **stand-alone technical report** suitable for academic submission or viva-voce examination, in the spirit of the course’s stated evaluation rubric (dataset & EDA, preprocessing & feature engineering, regression track, classification Part A, presentation, and viva).

- As a **reference document** for future contributors to the repository who need to understand design decisions before extending the work in Review 2.
- As a **teaching resource** that explains, in reasonable depth, why each modelling and evaluation choice was made, not merely what was done.

1.3 Repository Structure

The repository is organized around a simple and conventional layout for a two-notebook machine-learning project:

```
.
|-- README.md
|-- requirements.txt
|-- data/
|   |-- housing.csv
|   |-- adult.csv
|-- notebooks/
|   |-- regression.ipynb
|   |-- classification.ipynb
|-- models/
|   |-- regression_best_model.pkl
|   |-- classification_best_model.pkl
```

At the point of writing, the version-controlled repository contains the top-level documentation (`README.md`) and the dependency manifest (`requirements.txt`); the `data/`, `notebooks/`, and `models/` directories described above are the working layout that the notebooks assume and populate at run time. In particular, the `models/` directory is not checked in permanently – it is created automatically the moment each notebook serializes its selected best model with `joblib`. This is a sensible engineering choice: trained model binaries are derived artifacts, not source, and re-generating them by re-running the notebooks guarantees that the saved model always corresponds to the code that produced it.

1.3.1 Software Environment and Dependencies

The project is implemented entirely in Python and targets Python 3.9 or later. The dependency manifest, reproduced in Table 1.1, reflects a standard, minimal data-science stack: no deep-learning framework is required because every algorithm used in Review 1 is a classical statistical or tree-based learner drawn from `scikit-learn`.

Table 1.1: Project dependencies as declared in `requirements.txt`

Package	Minimum Version
numpy	1.24.0
pandas	2.0.0
scikit-learn	1.3.0
matplotlib	3.7.0
seaborn	0.12.0
joblib	1.3.0
jupyter	1.0.0
nbconvert	7.0.0

The roles of these packages within the pipeline are as follows: `numpy` and `pandas` provide array and tabular data structures; `scikit-learn` supplies every estimator, preprocessing transformer, cross-validation utility, and evaluation metric used in the project; `matplotlib` and `seaborn` generate the exploratory and diagnostic visualizations; `joblib` serializes the final fitted models to disk; and `jupyter/nbconvert` support interactive notebook development and (optionally) conversion of the notebooks to static formats such as HTML or PDF for submission.

1.3.2 Execution Workflow

The repository documents a conventional, reproducible setup procedure: clone the repository, create an isolated virtual environment, install the pinned dependencies via `pip install -r requirements.txt`, and then execute each notebook *from top to bottom* so that every cell – data loading, cleaning, preprocessing, feature engineering, model training, evaluation, plotting, and model serialization – runs in the correct order. Running notebooks out of order, or restarting the kernel mid-way, is explicitly discouraged because several preprocessing steps (fitting scalers and encoders) create state that later cells depend upon.

1.4 Course Context: The 23CSE301 Capstone Rubric

The 23CSE301 capstone guidelines specify an algorithm checklist and an evaluation rubric spanning several categories: dataset selection and exploratory analysis, preprocessing and feature engineering, the regression track (ten algorithms), the classification track (ten algorithms, split into two parts of five), the clustering track (multiple algorithms with internal validity metrics), a presentation component, and a viva-voce examination. Review 1 corresponds to the midpoint of this checklist: the entire regression track plus the first five classification algorithms must be complete and demonstrably correct, with the remaining classification algorithms and the clustering track deferred to Review 2. Section 5.4 of this report cross-references every rubric item against what has actually been delivered in the repository.

Chapter 2

Datasets

Two publicly available, widely studied benchmark datasets underpin the Review 1 deliverables. Both are tabular datasets of moderate size, well suited to classical machine-learning algorithms and to the pedagogical goal of comparing many algorithms under identical, controlled conditions.

2.1 California Housing Dataset (Regression Track)

2.1.1 Description

The California Housing dataset is a classic regression benchmark derived from the 1990 U.S. Census. Each row represents a *block group* – the smallest geographical unit for which the Census Bureau publishes sample data – and aggregates demographic, geographic, and housing statistics for that block group.

Table 2.1: California Housing dataset summary

Property	Value
Number of records	20,640
Raw columns	10
Numerical features	9
Categorical features	1 (ocean_proximity)
Target variable	median_house_value
Target range	\$14,999 – \$500,001
Missing values	207 (in total_bedrooms)

The nine numerical predictors describe geographic coordinates (longitude, latitude), the age of the housing stock (housing_median_age), counts of rooms and bedrooms (total_rooms, total_bedrooms), population and household counts (population, households), and the median income of residents (median_income). The single categorical predictor, ocean_proximity, records a coarse categorical label for the block group’s proximity to the coast (for example, ‘NEAR BAY’, ‘INLAND’, ‘<1H OCEAN’). The regression target, median_house_value, is the median value of owner-occupied housing units in the block group, capped at \$500,001 – a well-documented artifact of the original Census data-collection procedure that truncates all higher values into a single top bucket.

2.1.2 Data Quality Considerations

The only missing-data issue in this dataset is confined to a single column: `total_bedrooms` has 207 missing entries out of 20,640 rows (approximately 1% of records). Because this is a small, randomly distributed gap in a single numerical column, median imputation – rather than row deletion or more elaborate model-based imputation – is an appropriate and standard choice, and this is exactly the strategy adopted in the preprocessing pipeline (see Section 3.3).

The capped target ceiling at \$500,001 deserves particular attention during evaluation: it means that a small subset of the most expensive block groups are recorded with an artificially identical value rather than their true market value, which introduces an irreducible source of error for any model trying to predict housing value beyond that ceiling. This is a known limitation of the dataset itself, not a flaw introduced by the modelling pipeline, and it partially explains why even the best-performing model in this report does not achieve a near-perfect R^2 .

2.2 Adult / Census Income Dataset (Classification Track)

2.2.1 Description

The Adult dataset (also known as the “Census Income” dataset) is one of the most heavily used benchmarks in the classification literature. It originates from the 1994 U.S. Census Bureau database and poses a binary classification problem: predict whether an individual’s annual income exceeds \$50,000.

Table 2.2: Adult / Census Income dataset summary

Property	Value
Number of records	32,561
Raw columns	15
Target variable	<code>income (<=50K / >50K)</code>
Class balance	$\approx 76\%$ <code><=50K</code> , $\approx 24\%$ <code>>50K</code>
Missing-value marker	the literal string <code>"?"</code>
Affected columns	<code>workclass, occupation, native.country</code>

The predictor set spans demographic attributes (`age`, `sex`, `race`, `native.country`), socio-economic attributes (`workclass`, `education`, `education.num`, `occupation`), family-structure attributes (`marital.status`, `relationship`), financial attributes (`capital.gain`, `capital.loss`), and labour-market attributes (`hours.per.week`). The binary target, `income`, is notably imbalanced: roughly three-quarters of individuals in the dataset earn `<=50K` and one-quarter earn `>50K`. This imbalance is a first-order consideration for the entire classification track: it motivates the use of a *stratified* train/test split (so that both classes are proportionally represented in both partitions) and the use of *weighted* precision, recall, and F1-score as headline metrics rather than unweighted (macro) variants or raw accuracy alone, since a naive classifier that always predicts the majority class would already achieve roughly 76% accuracy without learning anything useful.

2.2.2 Data Quality Considerations

Unlike the California Housing dataset, where missingness is represented by the standard mechanism (empty/NaN cells), the Adult dataset encodes missing values using the literal placeholder string `"?"` in three categorical columns: `workclass`, `occupation`, and `native.country`. If this placeholder is not explicitly recognized and converted, it will silently be treated as a valid category by any

downstream encoder, corrupting both the exploratory analysis and the trained model. The classification pipeline explicitly converts every occurrence of "?" to a proper NaN value before any imputation or encoding step is applied, which is a necessary and often-overlooked data-cleaning step for this particular dataset.

2.3 Why These Two Datasets?

The choice of the California Housing dataset for regression and the Adult dataset for classification is well justified pedagogically. Both are of a manageable size (tens of thousands of rows) that trains quickly even for computationally expensive algorithms such as Support Vector Machines and Gradient Boosting, yet both are large enough to yield statistically meaningful comparisons between ten (or five) different algorithm families. Both datasets mix numerical and categorical features, both contain a nontrivial and realistic missing-data problem, and both have a large, mature body of published benchmark results against which the project's own results can be sanity-checked. Critically, neither dataset is trivially separable or predictable, so the differences in performance across algorithm families – and the gains achieved by hyperparameter tuning – are genuine and instructive rather than artifacts of a toy problem.

Chapter 3

Regression Track: California Housing Price Prediction

3.1 Problem Statement

The regression task is to predict the median house value, `median_house_value`, of a California block group given its demographic, geographic, and housing-related attributes. This is a canonical supervised regression problem: the target is a continuous, strictly positive quantity, and the objective is to minimize prediction error on unseen block groups while avoiding overfitting to the 20,640 training examples available.

3.2 Exploratory Data Analysis

Before any model is trained, the regression notebook performs a structured exploratory data analysis (EDA) pass whose purpose is to surface data-quality issues, understand feature distributions, and generate hypotheses about which features are likely to be predictive. The EDA proceeds through the following stages:

1. **Shape and data-type inspection.** The notebook first confirms the dataset's dimensions (20,640 rows by 10 columns) and the inferred data type of each column, distinguishing the nine numerical columns from the single categorical column `ocean_proximity`.
2. **Missing-value audit.** A per-column count of missing values confirms that `total_bedrooms` is the only column with gaps (207 missing entries), and that every other column is fully populated.
3. **Summary statistics.** Descriptive statistics (mean, standard deviation, quartiles, minimum, and maximum) are computed for every numerical column, giving a first indication of scale differences between features – for example, `median_income` is expressed in tens of thousands of dollars while `population` is expressed in raw headcounts, immediately motivating the need for feature scaling before applying distance- or gradient-based algorithms.
4. **Target distribution analysis.** A histogram of `median_house_value` reveals a right-skewed distribution with a pronounced spike at the \$500,001 ceiling, corroborating the census data-capping artifact discussed in Section 2.1.
5. **Feature distribution and skewness analysis.** Histograms of each numerical predictor reveal that several count-based features (`total_rooms`, `total_bedrooms`, `population`, `households`) are heavily right-skewed, since a small number of unusually large or dense block groups pull the distribution's tail outward.

6. **Feature-target relationship visualizations.** Scatter plots of each numerical feature against `median_house_value` are used to visually assess the strength and shape of each bivariate relationship, most notably confirming a strong, roughly linear positive relationship between `median_income` and the target.
7. **Ocean-proximity versus house-value analysis.** A grouped comparison (e.g., box plots) of `median_house_value` across the categories of `ocean_proximity` shows that block groups located inland tend to have systematically lower median house values than those near the coast or near the bay, motivating the inclusion of this categorical feature in the model despite it requiring encoding.
8. **Correlation analysis.** A correlation matrix (typically visualized as a heatmap) quantifies the linear association between all numerical features and the target, confirming `median_income` as the single strongest linear predictor and revealing expected multicollinearity between the four count-based features (`total_rooms`, `total_bedrooms`, `population`, `households`), since all four scale together with the physical size of a block group.

Each of these analytical steps in the notebook is accompanied by written observations that interpret the visualization rather than simply presenting it, which is precisely the kind of interpretive narrative that distinguishes a genuine EDA from a mechanical plotting exercise.

3.3 Data Preprocessing

The regression pipeline follows preprocessing best practice by strictly separating the training and test partitions before any transformation is fitted, thereby preventing data leakage. Concretely, the pipeline performs:

- An **80:20 train/test split**, so that 20% of the 20,640 block groups (approximately 4,128 rows) are held out purely for final evaluation and never seen during training or preprocessing fitting.
- **Median imputation** of the 207 missing values in `total_bedrooms`. Median imputation is preferred over mean imputation here because the count-based housing features are right-skewed, and the median is far more robust to the influence of the small number of very large block groups that stretch the distribution's tail.
- **Standardization** of all numerical features via `StandardScaler`, which subtracts each feature's training-set mean and divides by its training-set standard deviation. This step is essential for any algorithm sensitive to feature scale – in this project, most importantly the Support Vector Regressor and the K-Nearest Neighbors Regressor, both of which compute distances or dot products directly in feature space, but it is harmless (and conventional) to apply to all numerical features uniformly, including for the linear and regularized linear models.
- **One-hot encoding** of `ocean_proximity` via `OneHotEncoder(drop='first')`. Dropping the first category avoids the “dummy variable trap” – perfect multicollinearity between the one-hot columns and an implicit intercept term – which is particularly important for the linear-family models (Linear, Ridge, Lasso, ElasticNet) whose coefficient estimates and standard errors can otherwise become ill-defined or unstable.
- Crucially, **all of the above transformations are fit exclusively on the training partition** and then merely *applied* (via `transform`, not `fit_transform`) to the test partition. This guarantees that no information from the held-out test set – not even something as innocuous as its mean or its set of observed categories – leaks into the fitted preprocessing objects, which would otherwise produce an optimistic bias in the reported test-set metrics.

3.4 Feature Engineering

Beyond the raw census variables, the regression notebook constructs three domain-motivated ratio features that are well known in the California Housing literature to improve predictive performance:

$$\text{rooms_per_household} = \frac{\text{total_rooms}}{\text{households}} \quad (3.1)$$

$$\text{bedrooms_per_room} = \frac{\text{total_bedrooms}}{\text{total_rooms}} \quad (3.2)$$

$$\text{population_per_household} = \frac{\text{population}}{\text{households}} \quad (3.3)$$

These derived features are more informative than their raw components in isolation, because the raw counts (`total_rooms`, `total_bedrooms`, `population`, `households`) are all confounded with the physical size of the block group – a large block group naturally has more of everything. Normalizing each count by the number of households removes this size confound and instead captures genuinely informative per-household characteristics: how spacious the average home is, what fraction of rooms are bedrooms (a proxy for dwelling type and comfort level), and how crowded the average household is. As reported in Section 3.11, `population_per_household` indeed emerges as one of the more influential features in the final tuned model, validating this feature-engineering decision empirically rather than merely by intuition.

3.5 Algorithms Implemented

The Review 1 rubric requires ten regression algorithms to be implemented, trained, and compared on an identical held-out test set. All ten are present in the repository’s regression notebook:

1. **Linear Regression** – ordinary least squares, the baseline linear model with no regularization.
2. **Ridge Regression** – linear regression with an ℓ_2 penalty on the coefficients, shrinking them toward zero to reduce variance at the cost of a small amount of bias.
3. **Lasso Regression** – linear regression with an ℓ_1 penalty, which additionally performs implicit feature selection by driving some coefficients exactly to zero.
4. **ElasticNet Regression** – a convex combination of the ℓ_1 and ℓ_2 penalties, combining Lasso’s sparsity with Ridge’s stability in the presence of correlated features.
5. **Polynomial Regression** – linear regression applied to an expanded feature set including polynomial (and typically interaction) terms of the original features, allowing the model to capture curvature that a purely linear model cannot.
6. **Decision Tree Regressor** – a single recursive-partitioning tree that splits the feature space into axis-aligned regions and predicts the mean target value within each region.
7. **Random Forest Regressor** – an ensemble of many decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split, whose predictions are averaged to reduce variance.
8. **Gradient Boosting Regressor** – an additive ensemble of shallow decision trees trained sequentially, where each new tree is fit to the residual errors of the ensemble so far, gradually reducing bias.

9. **Support Vector Regressor (SVR)** – an extension of the support-vector-machine framework to regression, which fits a function that deviates from each training target by no more than a chosen margin (ϵ), while remaining as flat as possible.
10. **K-Nearest Neighbors Regressor** – a non-parametric method that predicts the target for a new point as the (possibly weighted) average of the targets of its k nearest neighbors in feature space.

3.5.1 Algorithmic Rationale

These ten algorithms are not an arbitrary list; together they span the major algorithmic families used in applied regression and allow the project to draw meaningful conclusions about which family of methods is best suited to this particular dataset:

- **Linear family** (Linear, Ridge, Lasso, ElasticNet, Polynomial): these models assume (after any polynomial expansion) an additive linear relationship between features and target. They are fast, interpretable, and provide a baseline against which more flexible models can be judged. Ridge, Lasso, and ElasticNet additionally guard against overfitting when features are correlated (as several census features are, per the correlation analysis in Section 3.2).
- **Tree-based family** (Decision Tree, Random Forest, Gradient Boosting): these models make no linearity assumption and can capture arbitrary nonlinear interactions between features. A single decision tree is prone to high variance (overfitting the training data), which the ensembling strategies of Random Forest (bagging) and Gradient Boosting (sequential residual fitting) are specifically designed to mitigate, each via a different variance/bias trade-off mechanism.
- **Instance-based method** (KNN): this model makes no explicit parametric assumption at all, instead deferring the entire “model” to the training data itself and computing predictions from local neighborhoods at inference time.
- **Kernel method** (SVR): this model can, in principle, capture nonlinear relationships through the kernel trick, while retaining a convex, globally optimal training objective – in contrast to the tree ensembles, whose training objectives are optimized greedily.

All ten models are trained on the identical, feature-engineered, preprocessed training partition, and every model is scored on the identical held-out test partition, which is what makes the comparison in Section 3.7 valid and fair.

3.6 Evaluation Metrics

Three complementary regression metrics are reported for every model:

- **Coefficient of Determination (R^2)**: the proportion of variance in the target explained by the model, defined as

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

An R^2 of 1.0 indicates perfect prediction; an R^2 of 0.0 indicates the model performs no better than always predicting the training-set mean; a negative R^2 indicates the model performs *worse* than that trivial baseline.

- **Root Mean Squared Error (RMSE)**: the square root of the average squared prediction error, expressed in the same units as the target (US dollars), which penalizes large errors more heavily than small ones due to the squaring operation.
- **Mean Absolute Error (MAE)**: the average absolute prediction error, also expressed in dollars, which is more robust to a small number of very large errors (outliers) than RMSE.

Reporting all three metrics together gives a fuller picture than any single metric alone: a model with a low MAE but high RMSE, for instance, is making mostly small errors punctuated by occasional very large ones, which would not be apparent from MAE alone.

3.7 Regression Results

Table 3.1 reports the test-set performance of all ten regression algorithms, each trained with its default (untuned) hyperparameters and evaluated on the same held-out 20% test partition.

Table 3.1: Baseline test-set performance of all ten regression algorithms

Model	R^2	RMSE (\$)	MAE (\$)
Linear Regression	0.6353	69,127.04	49,645.49
Ridge Regression	0.6352	69,136.26	49,653.25
Lasso Regression	0.6348	69,179.21	49,648.56
ElasticNet Regression	0.6287	69,757.85	50,091.85
Polynomial Regression	0.6209	70,480.08	44,709.93
Decision Tree Regressor	0.7073	61,928.77	41,936.01
Random Forest Regressor	0.7963	51,668.88	33,800.13
Gradient Boosting Regressor	0.7807	53,612.12	36,523.02
Support Vector Regressor (SVR)	-0.0871	119,355.55	101,782.44
KNN Regressor	0.7120	61,429.77	40,669.99

3.7.1 Interpretation of Baseline Results

Several clear patterns emerge from Table 3.1:

- **The linear family clusters tightly together** at $R^2 \approx 0.62$ – 0.64 . Linear Regression, Ridge, and Lasso perform almost identically, which is expected: with a moderate number of features (13 after encoding and feature engineering) and a training set of over 16,000 rows, the regularization penalties in Ridge and Lasso have relatively little room to improve on unregularized least squares, since overfitting is not a severe risk at this sample-to-feature ratio. ElasticNet, which blends both penalties, performs marginally worse still, likely because neither pure ℓ_1 nor pure ℓ_2 shrinkage is strongly needed, so the additional regularization simply introduces a small amount of extra bias without a corresponding variance benefit. Polynomial Regression, despite its added flexibility, does not meaningfully outperform plain Linear Regression in R^2 terms, though it does noticeably reduce MAE, suggesting that a small amount of curvature capture helps typical-case predictions without resolving the dataset’s largest sources of error.
- **Tree-based ensembles dominate.** Random Forest (0.7963) and Gradient Boosting (0.7807) are the two strongest baseline models by a wide margin, each explaining roughly 15–16 percentage points more variance than the best linear model. This is consistent with the well-documented

tendency of the California Housing dataset to contain nonlinear and interaction effects (for example, the value of `median_income` likely interacts with `ocean_proximity`) that tree-based partitioning can capture but a linear model, even with polynomial terms, cannot easily represent without extensive manual interaction engineering.

- **A single Decision Tree underperforms its ensembles**, as expected: an individual, unconstrained tree tends to overfit the training data, producing a test-set R^2 (0.7073) well below either ensemble method that averages or sequentially corrects many trees.
- **KNN performs respectably** (0.7120), confirming that local neighborhood averaging captures a meaningful amount of the dataset’s nonlinear structure, though it still falls short of the tree ensembles.
- **SVR fails outright under default settings**, producing a *negative* R^2 of -0.0871 – worse than simply predicting the training-set mean for every test example. This is a well-known pitfall of applying SVR with default hyperparameters (in particular, the default regularization parameter $C = 1.0$ and the default kernel width) to a target with a wide numeric range: without careful tuning of C , ϵ , and the kernel parameters, the default SVR is far too constrained to fit a target spanning tens of thousands of dollars, even after the features themselves have been standardized. This result is reported honestly in the notebook rather than hidden, which is itself methodologically important: an ML report should show where a technique fails, not just where it succeeds.

Based purely on this baseline comparison, **Random Forest Regressor** is identified as the best-performing model, with $R^2 = 0.7963$, RMSE of \$51,668.88, and MAE of \$33,800.13.

3.8 Cross-Validation

Baseline comparison on a single train/test split, while informative, can be misleading if that particular split happens to be unusually favorable or unfavorable for a given model. To obtain a more statistically reliable estimate of generalization performance, the two strongest baseline models – Random Forest and Gradient Boosting – are subjected to **5-fold cross-validation** on the training partition.

In 5-fold cross-validation, the training data is partitioned into five equally sized folds; the model is trained five times, each time on four folds and validated on the remaining fold, and the five resulting validation scores are averaged. This procedure uses every training example for both fitting and validation (across different folds) without ever touching the held-out test set, giving a more robust estimate of expected performance and, via the standard deviation across folds, an indication of how sensitive that performance is to the particular subset of data used for training.

Table 3.2: 5-fold cross-validation results for the two strongest ensemble models

Model	Mean CV R^2	CV Std. Dev.
Random Forest	0.8006	0.0047
Gradient Boosting	0.7912	0.0057

Both models show very low standard deviation across folds (under 0.006), indicating that their performance is stable and not an artifact of a particularly lucky or unlucky train/test split. Notably, the cross-validated mean R^2 for both models (0.8006 and 0.7912) is slightly *higher* than their single-split baseline test scores (0.7963 and 0.7807), which is plausible sample variation rather than a methodological concern, since cross-validation and the single held-out test set are different (though overlapping) partitions of the data.

3.9 Hyperparameter Tuning

Following cross-validation, both ensemble models are tuned using **GridSearchCV**, an exhaustive search procedure that evaluates every combination of hyperparameter values within a specified grid, using cross-validation internally to score each combination, and returns the combination with the best mean cross-validated score.

3.9.1 Tuned Random Forest Regressor

The grid search identifies the following best hyperparameters for the Random Forest Regressor:

- `max_depth = None` (trees are grown until every leaf is pure or contains fewer samples than the minimum split threshold, i.e., depth is unconstrained)
- `min_samples_split = 2` (the default and most permissive setting, allowing a node to be split as long as it contains at least two samples)
- `n_estimators = 150` (the ensemble comprises 150 individual trees)

With these settings, the tuned Random Forest achieves a best 5-fold CV R^2 of **0.8113** and, on the held-out test set, an R^2 of **0.8121**, an RMSE of **\$49,616.99**, and an MAE of **\$31,861.38** – a clear improvement over both the untuned baseline and the un-tuned cross-validated estimate.

Somewhat counter-intuitively, the grid search favors an *unconstrained* tree depth and the most permissive possible split threshold. This is not itself a sign of overfitting for a Random Forest in the way it would be for a single Decision Tree, because each individual tree in the forest is trained on a different bootstrap resample and a different random subset of features at each split; the ensemble’s variance-reduction comes from *averaging across trees*, not from constraining any individual tree, so allowing each tree to grow fully deep and then relying on the ensemble average to smooth out the resulting high-variance individual predictions is a standard and often optimal configuration for Random Forests.

3.9.2 Tuned Gradient Boosting Regressor

The grid search identifies the following best hyperparameters for the Gradient Boosting Regressor:

- `learning_rate = 0.1` (each successive tree’s contribution to the ensemble is scaled by a factor of 0.1, a moderate value that balances convergence speed against the risk of overshooting)
- `max_depth = 6` (each individual boosting tree is a moderately deep, but still shallow-relative-to-the-forest-case, tree)
- `n_estimators = 150` (150 sequential boosting stages)

With these settings, the tuned Gradient Boosting model achieves a best 5-fold CV R^2 of **0.8326** and, on the held-out test set, an R^2 of **0.8366**, an RMSE of **\$46,267.30**, and an MAE of **\$30,189.77**.

Table 3.3: Tuned ensemble model performance on the held-out test set

Model	Test R^2	Test RMSE (\$)	Test MAE (\$)
Tuned Random Forest	0.8121	49,616.99	31,861.38
Tuned Gradient Boosting	0.8366	46,267.30	30,189.77

After tuning, **Gradient Boosting overtakes Random Forest** as the strongest model overall, a result consistent with the general tendency of gradient boosting methods to achieve slightly higher predictive accuracy than bagged random forests on tabular data once their (more numerous and more sensitive) hyperparameters – particularly learning rate, tree depth, and the number of boosting stages – are properly tuned, at the cost of typically longer training times and a greater risk of overfitting if tuning is done carelessly.

3.10 Final Model Selection

Using the highest test-set R^2 as the selection criterion, the project selects the **Tuned Gradient Boosting Regressor** as the final regression model, with:

- $R^2 = 0.8366$
- $\text{RMSE} = \$46,267.30$
- $\text{MAE} = \$30,189.77$

This represents an improvement of roughly 20 percentage points in explained variance over the best linear baseline (Linear Regression, $R^2 = 0.6353$), and a reduction in typical absolute prediction error (MAE) of nearly \$20,000 relative to that same baseline – a substantial and practically meaningful gain achieved through the combination of ensemble tree-based modelling and systematic hyperparameter tuning.

3.11 Model Diagnostics

Beyond the headline metrics, the regression notebook produces several diagnostic visualizations and statistics for the final tuned Gradient Boosting model:

- **Residual plot.** Plotting residuals (actual minus predicted value) against predicted value allows visual inspection of whether the model's errors are homoscedastic (constant variance) and centered around zero across the full range of predictions, or whether the model systematically over- or under-predicts in particular value ranges – most likely, given the target's known ceiling artifact, in the very highest price bracket.
- **Predicted-versus-actual plot.** Plotting predicted values against true values, with a reference diagonal line representing perfect prediction, gives an intuitive visual sense of how tightly the model's predictions track the ground truth and where the largest deviations occur.
- **Feature-importance visualization.** Gradient Boosting models expose an impurity-based feature-importance score for each input feature, summarizing how much each feature contributed, on average, to reducing prediction error across all trees in the ensemble.
- **Diagnostic statistics.** Summary statistics of the residual distribution (e.g., its mean, standard deviation, and range) quantify the extent to which the model's errors are well-behaved.

The feature-importance analysis identifies `median_income` as, by a clear margin, the single most influential feature in the fitted model – consistent with both economic intuition (income is a direct driver of what a household can afford to pay for housing) and the strong linear correlation with the target observed during EDA. The second and third most influential features are `ocean_proximity_INLAND`

(the one-hot indicator for inland block groups, reflecting the substantial price discount associated with distance from the coast) and the engineered feature `population_per_household` (reflecting household crowding as a proxy for both income level and demand pressure). The prominence of an engineered feature this high in the importance ranking is a direct validation of the feature-engineering work described in Section 3.4: a feature that did not exist in the raw dataset turned out to be among the three most useful predictors in the final, best-performing model.

Chapter 4

Classification Track – Part A: Income Prediction

4.1 Problem Statement

The classification task is to predict, for each individual in the Adult / Census Income dataset, whether their annual income falls into the category $\leq 50K$ or $> 50K$, based on demographic, educational, occupational, and financial attributes. This is a binary classification problem with a moderately imbalanced target distribution (approximately 76% / 24%), which shapes both the preprocessing strategy (stratified splitting) and the choice of evaluation metrics (weighted averages, ROC-AUC) discussed below.

4.2 Exploratory Data Analysis

The classification notebook's EDA pass mirrors the structure of the regression EDA but is adapted to a categorical target and a predominantly categorical feature set:

1. **Shape and data-type inspection**, confirming 32,561 records across 15 raw columns, with a mix of numerical (`age`, `education.num`, `capital.gain`, `capital.loss`, `hours.per.week`, and the Census sample-weighting column) and categorical (`workclass`, `education`, `marital.status`, `occupation`, `relationship`, `race`, `sex`, `native.country`) features.
2. **Missing-value audit**, identifying the placeholder "?" values concentrated in `workclass`, `occupation`, and `native.country`.
3. **Target class-balance analysis**, quantifying and visualizing the roughly 76%/24% split between the two income classes, which is the single most important EDA finding for shaping the rest of the classification pipeline.
4. **Feature distributions**, examining the shape of each numerical feature (for example, the heavily right-skewed `capital.gain` and `capital.loss` columns, in which the overwhelming majority of individuals report a value of zero) and the frequency of each category within the categorical features.
5. **Income versus education analysis**, cross-tabulating the `income` target against `education` (or the ordinal `education.num`) to confirm the intuitive and empirically well-established relationship that higher educational attainment is associated with a substantially greater proportion of individuals earning $> 50K$.

6. **Hours-per-week versus income analysis**, comparing the distribution of `hours.per.week` between the two income groups and finding that individuals earning $>50K$ tend to report higher median weekly working hours – an intuitive result reflecting both greater full-time/overtime employment and the kinds of salaried professional roles more likely to be associated with higher income.
7. **Numerical correlation analysis**, examining pairwise correlations among the numerical features to check for multicollinearity before model fitting.

4.3 Data Preprocessing

The classification preprocessing pipeline addresses several distinct data-quality challenges not present in the regression dataset:

- **Missing-value conversion.** Every occurrence of the placeholder string "?" in `workclass`, `occupation`, and `native.country` is first converted to a proper NaN value, since – as discussed in Section 2.4 – leaving this placeholder in place would cause it to be silently treated as a legitimate category.
- **Mode imputation for categorical features.** Once converted to NaN, missing categorical values are imputed using the *mode* (most frequent category) of each column, the standard imputation strategy for categorical data, where a numeric statistic such as the mean or median is not meaningful.
- **Median imputation for numerical features.** Any missing numerical values are imputed using the column median, following the same robustness rationale applied in the regression pipeline.
- **IQR-based capping for extreme numerical observations.** Numerical features prone to extreme outliers – most notably `capital.gain` and `capital.loss`, whose distributions are dominated by zeros punctuated by a small number of very large values – are capped using the interquartile-range (IQR) method: values falling below $Q_1 - 1.5 \times \text{IQR}$ or above $Q_3 + 1.5 \times \text{IQR}$ are capped to those respective bounds rather than removed, preserving the sample size while limiting the influence of extreme values on scale-sensitive downstream algorithms.
- **One-hot encoding** of all remaining categorical variables, converting each category into a binary indicator column suitable for input to every classification algorithm used in this project, none of which natively accepts string-valued categorical input.
- **Standardization** of numerical features via `StandardScaler`, again primarily benefiting the scale-sensitive algorithms in the comparison set (KNN and SVC).
- An **80:20 stratified train/test split** with `random_state=42`. Stratification ensures that the roughly 76%/24% class ratio observed in the full dataset is preserved, as closely as possible, in *both* the training and test partitions – an important safeguard against the possibility that a plain random split happens to produce a test set with a meaningfully different class balance than the training set, which would bias the evaluation.
- As in the regression pipeline, **every preprocessing transformation is fitted exclusively on the training partition** and merely applied to the test partition, preventing data leakage.

4.4 Feature Engineering

Two additional engineered features are introduced to better capture the financial signal embedded in the raw capital-gain and capital-loss columns:

$$\text{Capital_Net} = \text{capital.gain} - \text{capital.loss} \quad (4.1)$$

$$\text{Has_Capital_Gain} = \mathbb{I}[\text{capital.gain} > 0] \quad (4.2)$$

`Capital_Net` collapses the two raw columns into a single signed measure of net capital income, which is arguably more directly relevant to overall financial standing than either raw column alone – an individual with a large gain and a large offsetting loss has a very different net financial position than one with a large gain and no loss, even though the two raw `capital.gain` values might be identical. `Has_Capital_Gain` is a simple binary indicator that captures a qualitatively different signal: whether an individual has *any* capital gain at all, independent of its magnitude, which is informative because the mere presence of investment income is itself correlated with higher overall income and wealth, and because the raw `capital.gain` column’s extreme skew (the overwhelming majority of values being exactly zero) makes it difficult for some algorithms to exploit its information content directly without this kind of binary complement.

4.5 Algorithms Implemented

Classification Part A requires five algorithms, all present in the repository’s classification notebook:

1. **Logistic Regression** – a linear model that estimates the log-odds of the positive class as a linear function of the input features, trained by maximizing the likelihood of the observed labels; despite the name, it is a classification (not regression) algorithm.
2. **K-Nearest Neighbors (KNN)** – a non-parametric classifier that assigns each test point the majority class among its k nearest neighbors in (standardized) feature space; the repository uses $k = 7$.
3. **Gaussian Naive Bayes** – a probabilistic classifier that applies Bayes’ theorem under the (“naive”) assumption that all features are conditionally independent given the class label, and that each numerical feature is normally distributed within each class.
4. **Decision Tree Classifier** – a recursive-partitioning classifier analogous to the Decision Tree Regressor described in Chapter 3, but predicting a class label (or class probability) rather than a continuous value at each leaf; the repository constrains the tree to `max_depth=4` to control overfitting and to keep the resulting tree interpretable enough to visualize.
5. **Support Vector Classifier (SVC)** – the classification counterpart of SVR, which seeks the maximum-margin hyperplane (or, with a nonlinear kernel, the maximum-margin decision boundary in a transformed feature space) separating the two classes.

4.5.1 Algorithmic Rationale

As with the regression track, these five algorithms span meaningfully different modelling paradigms: **Logistic Regression** provides a fast, interpretable linear baseline whose coefficients can be directly related to odds ratios; **KNN** provides a non-parametric, purely local decision rule with no explicit training

phase; **Gaussian Naive Bayes** provides an extremely fast probabilistic baseline whose strong independence assumption is almost certainly violated by real Census data (education, occupation, and hours worked are clearly correlated) but which nonetheless serves as an instructive lower-bound comparison; **Decision Tree** provides an interpretable, nonlinear, rule-based classifier; and **SVC** provides a margin-based classifier capable, in principle, of nonlinear decision boundaries via its kernel.

4.6 Evaluation Metrics

Given the class imbalance identified during EDA, the classification notebook reports a comprehensive suite of metrics rather than relying on accuracy alone:

- **Accuracy** – the overall fraction of correctly classified test examples. Because roughly 76% of the test set belongs to the majority class, accuracy alone can be misleadingly high even for a model that performs poorly on the minority (>50K) class, which is exactly why the remaining metrics are needed.
- **Weighted Precision, Recall, and F1-score** – precision, recall, and F1 computed independently for each class and then averaged, weighted by each class’s support (number of true instances) in the test set. Weighting by support gives a fairer overall summary than simple (macro) averaging when the classes are imbalanced, since it reflects the actual class distribution the model will be deployed against, while still accounting for per-class performance rather than only the aggregate accuracy.
- **ROC-AUC** – the area under the Receiver Operating Characteristic curve, which plots the true-positive rate against the false-positive rate across all possible classification thresholds. ROC-AUC summarizes a model’s ability to *rank* positive examples above negative examples, independent of any single fixed decision threshold, and is a standard metric of choice for imbalanced binary classification problems.

The notebook additionally reports **confusion matrices** for every model, a **class-wise classification report** for the selected best model, and **overlayed ROC curves** for all five models on a single plot, allowing direct visual comparison of their ranking ability across the full range of decision thresholds. Because SVC does not, by default, produce calibrated class probabilities, the notebook uses the SVC’s raw *decision function* output (the signed distance from the separating hyperplane) as the ranking score for its ROC curve – a standard and appropriate substitution when probability estimates are unavailable or would require an additional, computationally expensive calibration step (Platt scaling) to obtain.

4.7 Classification Results

Table 4.1 reports the test-set performance of all five classification algorithms.

4.7.1 Interpretation of Results

Several noteworthy patterns emerge:

- **SVC and Logistic Regression are essentially tied** for the best overall performance, with SVC narrowly leading on weighted F1-score (0.8284 vs. 0.8281) while Logistic Regression achieves the highest ROC-AUC of any model (0.8832). This near-tie between a linear probabilistic model and a margin-based kernel model suggests that the decision boundary between the two income

Table 4.1: Test-set performance of all five Classification Part A algorithms

Model	Acc.	W. Prec.	W. Rec.	W. F1	ROC-AUC
SVC	0.8360	0.8273	0.8360	0.8284	0.8673
Logistic Regression	0.8348	0.8265	0.8348	0.8281	0.8832
KNN ($k = 7$)	0.8115	0.8097	0.8115	0.8106	0.8486
Decision Tree (depth = 4)	0.8213	0.8111	0.8213	0.8003	0.8620
Gaussian Naive Bayes	0.5045	0.8017	0.5045	0.5159	0.7010

classes, in this feature space, is close to linear – Logistic Regression’s strong ROC-AUC, achieved with a purely linear decision function, supports this interpretation, since a strongly nonlinear true boundary would typically allow a kernel method to pull well ahead in ranking ability.

- **KNN and Decision Tree occupy a middle tier**, both comfortably ahead of Naive Bayes but a few points behind the top two models. The shallow depth constraint on the Decision Tree (`max_depth=4`), chosen deliberately to keep the tree interpretable and visualizable, likely trades away some predictive accuracy relative to an unconstrained tree in exchange for that interpretability – a reasonable trade-off for Part A of the classification track, where the goal is breadth of algorithm coverage rather than squeezing out maximum single-model performance.
- **Gaussian Naive Bayes underperforms dramatically** on accuracy (0.5045, barely better than a coin flip) and weighted F1 (0.5159), despite a respectable weighted precision (0.8017) and a ROC-AUC (0.7010) that, while the lowest of the five models, is still well above chance level (0.5). This pattern – reasonable ranking ability (ROC-AUC) alongside very poor accuracy at the default threshold – is a classic symptom of a poorly calibrated probability model: Naive Bayes’s independence assumption is severely violated by the strongly correlated Census features (for instance, `education` and `education.num` are near-duplicates of each other, and `marital.status`, `relationship`, and `sex` are all mutually informative), which causes the model’s estimated class probabilities to be systematically miscalibrated even though the underlying *ranking* of examples by estimated probability remains reasonably informative. In effect, the model’s default 0.5 decision threshold ends up poorly matched to the true, imbalanced class distribution given these miscalibrated probabilities, dragging accuracy and F1 down sharply while ROC-AUC – which is threshold-independent – remains comparatively more resilient.

4.8 Best Model Selection

The notebook selects the classification model with the highest **weighted F1-score** as the Part A winner, since weighted F1 balances precision and recall while still accounting for class imbalance – a more suitable single-number criterion for this dataset than raw accuracy. By this criterion, the best Part A model is the **Support Vector Classifier (SVC)**:

- Accuracy: **0.8360**
- Weighted Precision: **0.8273**
- Weighted Recall: **0.8360**
- Weighted F1: **0.8284**
- ROC-AUC: **0.8673**

4.8.1 Class-wise Performance of the Best Model

Table 4.2 breaks down the SVC’s performance separately for each of the two income classes.

Table 4.2: Per-class classification report for the selected SVC model

Class	Precision	Recall	F1-score
$\leq 50K$	0.8656	0.9281	0.8958
$> 50K$	0.7069	0.5459	0.6160

This class-wise breakdown reveals an important nuance hidden by the aggregate weighted metrics: the model performs substantially better on the majority $\leq 50K$ class ($F1 = 0.8958$) than on the minority $> 50K$ class ($F1 = 0.6160$). In particular, the recall for the $> 50K$ class is only 0.5459, meaning the model correctly identifies barely more than half of all individuals who actually earn above \$50,000 – it misses nearly 45% of true high earners, classifying them as $\leq 50K$. This is a direct and expected consequence of the underlying class imbalance (only $\approx 24\%$ of examples belong to this class): even with a stratified split and standardized features, a classifier optimized for overall weighted performance will naturally lean toward the majority class unless explicitly rebalanced (for example, via class-weighting, oversampling, or a custom decision threshold tuned specifically to maximize minority-class recall). This is an important and honest limitation to note in the report: the headline accuracy and weighted F1 figures, taken alone, understate how much harder the model finds the minority class, and any real-world deployment decision (for instance, using this model to flag applicants for a loan or benefit program keyed to income) would need to account for this asymmetric performance explicitly.

4.9 Additional Diagnostic Outputs

Beyond the summary tables above, the classification notebook generates a comprehensive set of diagnostic visualizations for the full Part A model suite:

- **Confusion matrices for all five models**, allowing direct visual comparison of the specific error patterns (false positives versus false negatives) made by each algorithm, not just their aggregate summary statistics.
- **Overlaid ROC curves**, plotting all five models’ true-positive-rate versus false-positive-rate trade-off curves on a single set of axes, visually corroborating the ROC-AUC ranking in Table 4.1 and making clear just how much further above the diagonal (chance) line Logistic Regression and SVC sit compared to Naive Bayes.
- **Decision tree visualization**, rendering the fitted, depth-4 Decision Tree Classifier as an explicit tree diagram showing the sequence of feature splits (for example, an early split on `marital.status` or `education.num`, both intuitively strong predictors of income) leading to each leaf’s predicted class, providing a genuinely interpretable artifact that the other four “black-box-er” models cannot easily offer.
- **Model performance comparison chart**, a bar or grouped-bar visualization summarizing all five models across all five metrics side by side, giving at-a-glance confirmation of the SVC/Logistic-Regression near-tie at the top and the Naive Bayes underperformance at the bottom.
- **Detailed classification report for the selected model**, expanding Table 4.2 with support counts and macro-averaged metrics alongside the weighted metrics already discussed.

Chapter 5

Cross-Track Summary and Model Selection

5.1 Consolidated Model Selection

Table 5.1 consolidates the final selected model, the selection criterion applied, and the achieved test-set performance for each of the two tracks completed in Review 1.

Table 5.1: Consolidated model selection summary across both tracks

Track	Selected Model	Selection Criterion	Test Performance
Regression	Tuned Gradient Boosting Regressor	Highest test R^2 after tuning	$R^2 = 0.8366$
Classification Part A	Support Vector Classifier (SVC)	Highest weighted F1	F1 = 0.8284

Both selection criteria are principled and appropriate to their respective problem types: R^2 directly measures the proportion of target variance explained in a continuous-output regression problem, while weighted F1 balances precision and recall in a way that is sensitive to the classification track’s known class imbalance, rather than allowing a model to appear strong purely by favoring the majority class.

5.2 Common Methodological Threads

Despite targeting different problem types, the two tracks share a set of consistent, deliberate methodological choices that reflect good machine-learning engineering practice throughout the repository:

- **Identical evaluation protocol across all competing algorithms within a track.** In both the regression and classification tracks, every candidate algorithm is trained on the same preprocessed training partition and scored on the same held-out test partition, making the resulting comparison tables valid, apples-to-apples comparisons rather than being confounded by different candidates being evaluated under different conditions.
- **Leakage-safe preprocessing.** In both tracks, every preprocessing transformation – imputers, scalers, and encoders alike – is fit exclusively on the training data and only ever *applied* (never re-fit) to the test data, a discipline that is easy to state but frequently violated in less careful project work, and whose violation would silently and artificially inflate every reported test-set metric.
- **Reproducibility via fixed random seeds.** A fixed `random_state=42` is used wherever randomness enters the pipeline (train/test splitting, and the internal randomness of tree-ensemble

algorithms), ensuring that anyone re-running the notebooks obtains numerically identical results, which is essential both for grading/verification purposes and for meaningful debugging during development.

- **Purposeful feature engineering grounded in domain reasoning**, rather than blind feature multiplication: each engineered feature in both tracks (the three housing ratios; `Capital_Net` and `Has_Capital_Gain`) is motivated by a specific hypothesis about what raw features fail to capture directly, and in the regression track this hypothesis is explicitly validated post hoc via the feature-importance analysis.
- **Multi-metric evaluation rather than reliance on a single number**. Both tracks report several complementary metrics (three for regression, five for classification) precisely because any single metric can mask important failure modes – as vividly illustrated by Gaussian Naive Bayes’s respectable ROC-AUC alongside its dismal accuracy, and by SVR’s baseline failure being visible in RMSE/MAE terms as well as in R^2 .
- **Systematic, not ad hoc, hyperparameter tuning**. Where tuning is performed (the two ensemble regressors), it is done via exhaustive grid search combined with cross-validation, rather than manual trial-and-error, producing a defensible, reproducible record of exactly which hyperparameter combinations were considered and why the winning combination was chosen.

5.3 Reproducibility and Data Leakage Prevention

Reproducibility and leakage prevention are treated as first-class concerns throughout the repository rather than afterthoughts, and are worth restating explicitly:

1. **Ordering of operations**. In every pipeline, the train/test split is performed *before* any preprocessing transformation is fit. This ordering is the single most important safeguard against data leakage: any transformation fit on the full dataset (including the test partition) would allow statistics from the test set – even something as seemingly innocuous as the overall mean or the set of observed categories – to influence how the training data is transformed, indirectly leaking test-set information into the model.
2. **Consistent application of fitted transformers**. Numerical imputers and scalers, and categorical encoders, are all fit once on the training partition and then applied identically to the test partition via their `transform` method, guaranteeing that both partitions are represented in exactly the same feature space.
3. **A single, shared held-out test set per track**. Within each track, every candidate algorithm is scored against the same held-out partition, so that differences in reported metrics reflect genuine differences in model quality rather than differences in which subset of data happened to be used for evaluation.
4. **Fixed random seeding**. The consistent use of `random_state=42` throughout both pipelines ensures bit-for-bit reproducibility of the split itself and of any algorithm with an internal source of randomness.

5.4 Capstone Requirements Coverage

Table 5.2 maps every Review 1 rubric requirement, as documented in the repository, onto its completion status.

Table 5.2: Review 1 capstone requirements coverage

Requirement	Status
Dataset audit and EDA	Completed
Missing-value handling	Completed
Encoding and scaling	Completed
Feature engineering	Completed
All 10 regression algorithms	Completed
Regression comparison table	Completed
Regression 5-fold cross-validation	Completed
Regression hyperparameter tuning	Completed
Regression diagnostics	Completed
Classification Part A – 5 algorithms	Completed
Classification comparison table	Completed
Classification confusion matrices	Completed
Classification ROC curves / ROC-AUC	Completed
Classification decision-tree visualization	Completed

Every item required for the Review 1 checkpoint is reported as complete. The overall Review 1 rubric allocates marks across six categories – Dataset & EDA, Preprocessing & Feature Engineering, the Regression Track, Classification Part A, Presentation, and Viva – and the repository’s documentation and this report’s analysis together address the first four of these categories directly, while the Presentation and Viva components necessarily depend on material (slides, verbal defense) that lies outside the scope of the code repository itself.

5.5 Review 2: Planned Work

The repository’s README explicitly scopes the work remaining for Review 2, which this report reproduces here for completeness and to make clear the intended trajectory of the project:

5.5.1 Classification Part B

Five additional classification algorithms are planned:

1. Random Forest Classifier
2. AdaBoost Classifier
3. Gradient Boosting Classifier
4. Bagging Classifier
5. MLP (Multi-Layer Perceptron) Classifier

Once complete, these will be combined with the five Part A algorithms already implemented to produce a **consolidated comparison of all ten classification algorithms** required by the capstone, followed by classification hyperparameter tuning (mirroring the `GridSearchCV` approach already applied to the regression ensembles) and final classification model selection.

It is worth noting that four of these five Part B algorithms are themselves ensemble methods (Random Forest, AdaBoost, Gradient Boosting, and Bagging), which – based on the strong showing of

Random Forest and Gradient Boosting in the regression track – can reasonably be expected to be strong contenders once implemented, particularly if their known sensitivity to class imbalance is addressed via class weighting or resampling, an issue this report’s analysis of the Part A results (Section 4.6.2) suggests deserves explicit attention.

5.5.2 Full Clustering Track

The clustering track, entirely unstated at Review 1, will require:

- **K-Means clustering**, a centroid-based partitioning algorithm requiring the number of clusters k to be specified in advance.
- **Agglomerative Hierarchical Clustering**, a bottom-up clustering method that iteratively merges the closest pair of clusters until a single cluster (or a specified number of clusters) remains, naturally producing a dendrogram.

Clustering quality will be assessed using three complementary internal validation metrics, none of which require ground-truth labels (appropriate, since clustering is inherently unsupervised):

- **Silhouette Score**, which measures how similar each point is to its own cluster compared to the nearest neighboring cluster, ranging from -1 (poor clustering) to $+1$ (well-separated clustering).
- **Davies-Bouldin Index**, which measures the average similarity between each cluster and its most similar other cluster, where *lower* values indicate better clustering (clusters are more distinct from one another).
- **Calinski-Harabasz Index**, which measures the ratio of between-cluster dispersion to within-cluster dispersion, where *higher* values indicate better-defined clusters.

The required visualizations for this track include an **elbow curve** (plotting within-cluster sum of squares against k to identify a reasonable number of clusters for K-Means), a **dendrogram** (visualizing the hierarchical merge structure produced by agglomerative clustering), and a **PCA 2D visualization** (projecting the high-dimensional feature space down to two principal components so that cluster assignments can be visually inspected on a scatter plot), along with an optional **t-SNE visualization** where applicable, which – unlike PCA’s linear projection – can reveal nonlinear cluster structure that a purely linear projection might obscure.

5.5.3 Final Documentation

Review 2 will also produce a final, integrated README and repository documentation set spanning all three tracks (regression, classification, and clustering) in a single, coherent submission.

5.6 Optional Future Extensions

Beyond the required Review 2 scope, the capstone guidelines permit optional bonus work, which the repository’s roadmap explicitly flags as candidates for later addition once the required work is complete:

- An **interactive graphical user interface**, built using a lightweight Python web-app framework such as Streamlit or Gradio, that would allow a non-technical user to input feature values through form fields and receive live predictions from the serialized regression and/or classification models.

- **Public deployment** of such an application (for example, to a cloud hosting platform), which would allow the models developed in this capstone to be demonstrated and used outside of the local Jupyter-notebook environment in which they were built.

These extensions are appropriately deferred: they add genuine end-user value but are not part of the core machine-learning methodology being assessed by the capstone rubric, and pursuing them prematurely – before the required Review 2 clustering and classification work is complete – would risk diverting effort away from the graded core requirements.

Chapter 6

Theoretical Background of Algorithms Used

This chapter collects concise theoretical background on every algorithm employed across both tracks, intended as a self-contained reference for readers who wish to understand the underlying mechanics of each method beyond the applied results reported in Chapters 3 and 4.

6.1 Linear-Family Regression Models

6.1.1 Ordinary Least Squares (Linear Regression)

Linear Regression models the target as a linear combination of the input features,

$$\hat{y} = \beta_0 + \sum_{j=1}^p \beta_j x_j,$$

with coefficients β chosen to minimize the residual sum of squares $\sum_i (y_i - \hat{y}_i)^2$ on the training data. It is unbiased under standard assumptions but can exhibit high variance when features are correlated or numerous relative to the number of training examples.

6.1.2 Ridge, Lasso, and ElasticNet

Ridge Regression augments the least-squares objective with an ℓ_2 penalty, $\lambda \sum_j \beta_j^2$, shrinking coefficients toward zero without setting any exactly to zero. Lasso Regression instead uses an ℓ_1 penalty, $\lambda \sum_j |\beta_j|$, which can drive some coefficients to exactly zero, performing implicit feature selection. ElasticNet combines both penalties, $\lambda \left(\alpha \sum_j |\beta_j| + \frac{1-\alpha}{2} \sum_j \beta_j^2 \right)$, inheriting Lasso’s sparsity alongside Ridge’s stability when features are highly correlated with one another.

6.1.3 Polynomial Regression

Polynomial Regression applies the same least-squares machinery to an expanded feature set that includes powers and, typically, interaction terms of the original features (for example x_1^2 , $x_1 x_2$), allowing a nominally “linear” model to fit curved relationships in the original feature space at the cost of a rapidly growing number of parameters as the polynomial degree increases.

6.2 Tree-Based Models

6.2.1 Decision Trees

A Decision Tree recursively partitions the feature space by choosing, at each node, the single feature and threshold that most reduces an impurity criterion – mean squared error for regression, or Gini impurity / entropy for classification – between the parent node and its two children. Predictions are made by traversing from the root to a leaf and returning that leaf’s mean target value (regression) or majority class (classification). Unconstrained trees are prone to overfitting because they can, in principle, grow until every training example occupies its own leaf.

6.2.2 Random Forest

A Random Forest trains many decision trees, each on an independent bootstrap resample of the training data and considering only a random subset of features at each split, then averages (regression) or majority-votes (classification) across all trees. This combination of bootstrap aggregation (*bagging*) and random feature selection decorrelates the individual trees, so that averaging their predictions substantially reduces variance relative to any single tree, without materially increasing bias.

6.2.3 Gradient Boosting

Gradient Boosting builds an additive ensemble of typically shallow trees *sequentially*: each new tree is trained to predict the negative gradient of the loss function with respect to the current ensemble’s predictions (for squared-error loss, this reduces to fitting each new tree to the current residuals), and its predictions are added to the ensemble scaled by a learning rate. This sequential, error-correcting construction tends to reduce bias more effectively than bagging, at the cost of greater sensitivity to hyperparameters such as the learning rate, tree depth, and number of boosting stages, and a correspondingly greater risk of overfitting if these are not tuned carefully.

6.3 Instance-Based and Kernel Methods

6.3.1 K-Nearest Neighbors

KNN makes no explicit training-time model fit at all; instead, for a query point, it identifies the k training points closest to it (typically by Euclidean distance in standardized feature space) and predicts the average target value (regression) or majority class (classification) among those neighbors. Its performance is highly sensitive to the choice of k and to feature scaling, since distance computations are dominated by whichever features happen to have the largest numeric range if left unstandardized.

6.3.2 Support Vector Machines (SVR and SVC)

Support Vector Regression seeks a function that deviates from every training target by no more than a tolerance ε , while remaining as “flat” (low-norm) as possible, with a regularization parameter C controlling the trade-off between flatness and tolerance for larger deviations. Support Vector Classification instead seeks the maximum-margin hyperplane separating the two classes, again governed by a regularization parameter C that trades off margin width against the number and severity of margin violations. Both variants can employ nonlinear kernels (radial basis function, polynomial, etc.) to implicitly map the input features into a higher-dimensional space where a linear separator/regressor in that space corresponds to a nonlinear function in the original feature space. Both are highly sensitive to their hyperparameters and to feature scaling, which explains the dramatic baseline failure of the default-hyperparameter SVR documented in Section 3.7.

6.4 Probabilistic Models

6.4.1 Gaussian Naive Bayes

Gaussian Naive Bayes applies Bayes’ theorem,

$$P(y \mid \mathbf{x}) \propto P(y) \prod_{j=1}^p P(x_j \mid y),$$

under the simplifying (“naive”) assumption that features are conditionally independent given the class, and that each numerical feature’s class-conditional distribution, $P(x_j \mid y)$, is Gaussian. Its speed and simplicity make it a useful baseline, but its accuracy suffers when the independence assumption is strongly violated, as observed for the Adult dataset in Section 4.6.1.

6.4.2 Logistic Regression

Logistic Regression models the log-odds of the positive class as a linear function of the input features,

$$\log \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \beta_0 + \sum_{j=1}^p \beta_j x_j,$$

with coefficients estimated by maximizing the likelihood of the observed training labels. Despite its linear decision boundary, it directly produces calibrated class probabilities and is often a surprisingly strong and highly interpretable baseline for binary classification problems, as demonstrated by its near-best performance in the Part A comparison.

6.5 Evaluation Concepts

6.5.1 R^2 , RMSE, and MAE

These three regression metrics, defined formally in Section 3.6, jointly characterize a regression model’s fit: R^2 expresses goodness-of-fit on a normalized, interpretable 0-to-1 (or negative) scale; RMSE expresses typical error magnitude in the target’s native units while penalizing large errors more heavily; and MAE expresses typical error magnitude in a way that is more robust to a small number of unusually large errors.

6.5.2 Precision, Recall, F1, and ROC-AUC

For a binary classifier, precision is the fraction of predicted-positive examples that are truly positive, and recall is the fraction of truly positive examples that are correctly predicted positive; F1 is their harmonic mean, balancing the two. Weighted variants of these three metrics average the per-class values, weighted by each class’s frequency in the test set, appropriately reflecting an imbalanced target distribution such as the Adult dataset’s 76%/24% split. ROC-AUC, by contrast, evaluates a classifier’s ranking ability across every possible decision threshold simultaneously, making it a threshold-independent complement to the threshold-dependent precision/recall/F1 family.

6.5.3 Cross-Validation and Grid Search

K -fold cross-validation partitions the training data into K folds, trains on $K - 1$ folds and validates on the remaining fold K times (rotating which fold is held out), and averages the resulting validation scores

to obtain a more robust estimate of expected out-of-sample performance than a single train/validation split provides. Grid search combines this idea with hyperparameter optimization: for every candidate combination of hyperparameter values in a specified grid, the model is cross-validated, and the combination achieving the best mean cross-validated score is selected as the tuned model's final configuration.

Chapter 7

Conclusion

7.1 Summary of Findings

This report has documented, in comprehensive detail, the Review 1 deliverables of the 23CSE301 Machine Learning Capstone Project as implemented in the `Review_1_ASV_reg_cla` repository. Two complete machine-learning pipelines were examined: a ten-algorithm regression pipeline predicting median house value on the California Housing dataset, and a five-algorithm classification pipeline (Part A) predicting income category on the Adult / Census Income dataset.

In the regression track, systematic comparison of ten algorithms across the major regression paradigms – linear, regularized-linear, polynomial, tree-based, instance-based, and kernel-based – established that tree-based ensembles substantially outperform every other family on this dataset, with the baseline Random Forest ($R^2 = 0.7963$) and Gradient Boosting ($R^2 = 0.7807$) models clearly ahead of the linear family ($R^2 \approx 0.62$ – 0.64) and comfortably ahead of a single Decision Tree ($R^2 = 0.7073$) and KNN ($R^2 = 0.7120$), while the default-configuration SVR failed entirely ($R^2 = -0.0871$). Cross-validation confirmed the stability of the two ensemble methods’ performance, and systematic hyperparameter tuning via grid search improved both further, with the tuned Gradient Boosting Regressor emerging as the final selected model at $R^2 = 0.8366$, RMSE of \$46,267.30, and MAE of \$30,189.77 – with feature-importance analysis confirming `median_income`, the engineered `population_per_household` ratio, and the inland-location indicator as the model’s most influential predictors.

In the classification track, SVC and Logistic Regression emerged as an essentially tied top pair, each achieving roughly 83.5% accuracy and weighted F1 scores around 0.828, with Logistic Regression achieving the single highest ROC-AUC (0.8832) among all five models – evidence that the underlying decision boundary between income classes is close to linear in the engineered feature space. Gaussian Naive Bayes’s severe accuracy shortfall (0.5045), despite a still-informative ROC-AUC (0.7010), illustrated the practical cost of its independence assumption when applied to strongly correlated Census features. The selected best model, SVC, was shown on closer class-wise inspection to perform substantially better on the majority $\leq 50K$ class (F1= 0.8958) than on the minority $> 50K$ class (F1= 0.6160, recall of only 0.5459) – an important, honestly reported limitation directly attributable to the dataset’s class imbalance.

7.2 Methodological Strengths

Across both tracks, the project consistently applies sound machine-learning methodology: leakage-safe preprocessing (fitting every transformer exclusively on training data), reproducible experimentation (fixed random seeds throughout), fair like-for-like model comparison (identical train/test partitions and preprocessing across all competing algorithms within a track), purposeful and empirically validated

feature engineering, multi-metric evaluation that avoids over-reliance on any single summary statistic, and systematic (rather than ad hoc) hyperparameter tuning via cross-validated grid search. These are precisely the practices that distinguish a rigorous, trustworthy machine-learning pipeline from a superficially similar one whose reported metrics may not generalize.

7.3 Limitations and Areas for Improvement

Several limitations, some inherent to the datasets and some addressable through further methodological refinement, are worth noting for completeness:

- The California Housing target’s ceiling artifact (values capped at \$500,001) introduces an irreducible source of error that no amount of modelling improvement can fully overcome, and ideally would be flagged even more prominently in any deployment of this model.
- The classification track’s minority-class recall shortfall (Section 4.6.2) suggests that class-imbalance-aware techniques – class weighting, oversampling methods such as SMOTE, or a custom decision threshold selected to optimize minority-class recall rather than the default 0.5 cutoff – would likely improve the practical usefulness of the classification models, and this is a natural candidate for methodological refinement in Review 2’s Part B work.
- The Decision Tree Classifier’s depth constraint (`max_depth=4`), while good for interpretability, was not itself subjected to hyperparameter tuning in Part A; a shallow-versus-tuned-depth comparison could clarify how much accuracy, if any, is being traded away for the current tree’s interpretability.
- The SVR’s baseline failure, while honestly and usefully reported, was not followed up with a tuned SVR configuration in the regression track (tuning effort was instead directed at the two strongest baseline models, Random Forest and Gradient Boosting) – a reasonable prioritization given limited effort, but one that leaves open the question of how competitive a properly tuned SVR might become.

7.4 Outlook Toward Review 2

The repository’s own roadmap, reproduced in Section 5.6, lays out a clear and achievable path to project completion: five additional classification algorithms (Part B), a consolidated ten-algorithm classification comparison and tuning pass, a full clustering track with three internal validity metrics and the accompanying elbow-curve, dendrogram, and dimensionality-reduction visualizations, and a final integrated project write-up. Given the methodological discipline already demonstrated in the Review 1 deliverables – careful leakage prevention, systematic comparison, and honest reporting of both successes and failures – there is every reason to expect that the Review 2 deliverables will meet the same standard of rigor.

References

1. California Housing dataset (derived from the 1990 U.S. Census), used for the regression track of this project.
2. Adult / Census Income dataset (1994 U.S. Census Bureau database), used for the classification track of this project.
3. 23CSE301 Machine Learning Capstone Project Guidelines, Algorithm List and Evaluation Rubrics, Academic Year 2026–27, Amrita Vishwa Vidyapeetham, Department of Computer Science and Engineering.
4. Pedregosa, F., et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2011. (Library used for all preprocessing transformers, estimators, cross-validation, and evaluation metrics in this project.)
5. McKinney, W. *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 2010. (`pandas` library, used for all data loading and manipulation.)
6. Harris, C.R., et al. *Array Programming with NumPy*. Nature, 585, 2020. (`numpy` library, used for underlying numerical computation.)
7. Hunter, J.D. *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 2007, and Waskom, M. *seaborn: statistical data visualization*. Journal of Open Source Software, 6(60), 2021. (Visualization libraries used throughout the exploratory and diagnostic analysis.)
8. Repository source: `Review_1_ASV_reg_cla`, GitHub user `ashwkumarsrinialg-cpu`.